

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

а. Реализовать алгоритм растровой развертки многоугольника (в соответствии с вариантом): А7. построчного заполнения с затравкой для восьмисвязной гранично-определенной области;

б. Реализовать алгоритм фильтрации (в соответствии с вариантом): Б2. постфильтрация с взвешенным усреднением области NxN

в. Реализовать необходимые вспомогательные алгоритмы (растеризации отрезка) с модификациями, обеспечивающими корректную работу основного алгоритма.

г. Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши. Предусмотреть также возможность очистки области вывода (отмены ввода).

д. Растеризацию производить в специально выделенном для этого буфере в памяти с последующим копированием результата в буфер кадра OpenGL. Предусмотреть возможность изменение размеров окна.

2 Основная теория

2.1 Целочисленный алгоритм Брезенхема

Алгоритм Брезенхема предназначен для растеризации отрезка, т.е. определения целочисленных координат пикселей, наиболее близких к идеальной прямой. Достоинство алгоритма – использование только целочисленных операций, что обеспечивает высокую скорость.

Рассматривается отрезок из точки (x_0, y_0) в (x_1, y_1). Для случая $|\Delta x| \geq |\Delta y|$ и $\Delta x > 0$ основное приращение делается по координате x. Вводится переменная ошибки e, которая на каждом шаге увеличивается на $2\Delta y$. Если ошибка становится неотрицательной, то координата y увеличивается на 1, а ошибка уменьшается на $2\Delta x$. Начальное значение $e_0 = -\Delta x$. Аналогично для случая $|\Delta y| > |\Delta x|$ приращение делается по y, а ошибка управляет изменением x.

2.2 Алгоритм построчного заполнения с затравкой

Для заливки замкнутой области, ограниченной пикселями граничного цвета, применяется построчный алгоритм с затравкой. В отличие от рекурсивного заполнения всех соседей, данный алгоритм сначала закрашивает целую строку между левой и правой границами, а затем обрабатывает строки выше и ниже, помещая в стек затравочные точки для новых сегментов.

Алгоритм работает следующим образом:

1. Поместить начальную затравку (x_0, y_0) в стек.
2. Пока стек не пуст:
   (a) Извлечь точку (x, y).
   (b) Найти левую границу x_{left} текущей строки, двигаясь влево от x, пока не будет достигнут граничный или уже заполненный пиксель.
   (c) Найти правую границу x_{right}, двигаясь вправо.
   (d) Закрасить все пиксели от $x_{\text{left}} + 1$ до $x_{\text{right}} - 1$ цветом заливки.
   (e) Для каждого закрашенного пикселя проверить пиксели в строках $y - 1$ и $y + 1$: если пиксель не является граничным и не закрашен, и он является началом непрерывного участка (либо самый левый, либо его левый сосед — граница или заливка), то добавить его в стек в качестве новой затравки.

2.3 Постфильтрация с равномерным усреднением

Фильтрация изображения применяется для сглаживания и подавления шумов. В работе реализован линейный фильтр с ядром 3×3, где каждый выходной пиксель вычисляется как среднее арифметическое значений пикселей в окрестности.

Для пикселя (i, j) новое значение цвета $I'(i, j)$ вычисляется по формуле:

$$I'(i, j) = \frac{1}{9} \sum_{k=-1}^{1} \sum_{l=-1}^{1} I(i + k, j + l),$$

где $I(i, j)$ – исходное значение. Для цветного изображения операция выполняется независимо для каждой компоненты RGB.

В представленной программе используется ядро 3×3 с весами:

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Такая маска даёт более естественное сглаживание с сохранением деталей.

2.4 Особенности реализации

Фильтрация применяется ко всему буферу после завершения рисования. Для обработки граничных пикселей (не имеющих полной окрестности) можно либо игнорировать их, либо обрабатывать по доступным соседям. В коде используется копирование исходного буфера и обработка пикселей, начиная со второй строки и столбца, заканчивая предпоследними.

2.5 Обработка изменения размера окна

При изменении размеров окна вызывается callback $\mathrm{framebuffer_size_callback}$, который обновляет глобальные переменные ширины и высоты, а также изменяет размер буфера. При этом старое содержимое копируется в левый верхний угол нового буфера, а новые области заполняются белым цветом. Такой подход сохраняет нарисованную фигуру.

2.6 Интерактивный ввод и управление

Программа поддерживает следующие действия:

- **Левая кнопка мыши** – добавление вершины многоугольника. Между последними двумя добавленными точками автоматически рисуется отрезок по алгоритму Брезенхема.

- **Правая кнопка мыши** – после построения хотя бы трёх вершин замыкает многоугольник и выполняет заливку области, используя точку под курсором в качестве затравки.
- **Клавиша C** – очистка буфера (заполнение белым цветом) и сброс списка вершин.
- **Клавиша F** – применение фильтра усреднения (равномерного или гауссова) ко всему изображению.

Все операции выполняются в реальном времени, буфер обновляется и отображается на экране.

3 Практическая реализация

Листинг 1: Основной код

```
from tkinter import NE

import glfw
from OpenGL.GL import *
import numpy as np

WINDOW_WIDTH = 800
WINDOW_HEIGHT = 600

class Polygon():
    def __init__(self):
        self.points = []
        self.edges_color = (255, 0, 0)
        self.fill_color = (0, 255, 0)
        self.buffer = np.full((WINDOW_HEIGHT, WINDOW_WIDTH, 3), 255, dtype=np.
    uint8)

    def clear_buffer(self):
        self.buffer.fill(255)
        self.points = []

    def add_point(self, x, y):
        self.points.append((x, y))

    def set_pixel(self, x, y, color):
```

```
25          if 0 <= x < self.buffer.shape[1] and 0 <= y < self.buffer.shape[0]:
26              self.buffer[y, x] = color
27
28      def is_edge_point(self, x, y):
29          return not (0 <= y < WINDOW_HEIGHT and 0 <= x < WINDOW_WIDTH) or
        tuple(self.buffer[y, x]) == self.edges_color
30
31      def is_filled_point(self, x, y):
32          return 0 <= y < WINDOW_HEIGHT and 0 <= x < WINDOW_WIDTH and tuple(
        self.buffer[y, x]) == self.fill_color
33
34      def is_free(self, x, y, x_left):
35          return not self.is_edge_point(x, y) and not self.is_filled_point(x, y) \
36                      and (x == x_left or self.is_edge_point(x - 1, y) or self.is_filled_point(x -
        1, y))
37
38      def bresenham(self, point1, point2):
39          x0, y0 = point1
40          x1, y1 = point2
41          dx = abs(x1 - x0)
42          dy = abs(y1 - y0)
43          sx = 1 if x0 < x1 else -1
44          sy = 1 if y0 < y1 else -1
45
46          if dx >= dy:
47              e = -dx
48              de = 2 * dy
49              while True:
50                  self.set_pixel(x0, y0, self.edges_color)
51                  if x0 == x1:
52                      break
53                  e += de
54                  x0 += sx
55                  if e >= 0:
56                      y0 += sy
57                      e -= 2 * dx
58          else:
59              e = -dy
60              de = 2 * dx
61              while True:
62                  self.set_pixel(x0, y0, self.edges_color)
63                  if y0 == y1:
64                      break
65                  e += de
66                  y0 += sy
67                  if e >= 0:
```

```
                    x0 += sx
                    e -= 2 * dy

    def fill_with_zatravka(self, x0, y0):
        stack = [(x0, y0)]
        while len(stack) != 0:
            x, y = stack.pop()
            x_left, x_right = x, x
            while not self.is_edge_point(x_left, y):
                self.set_pixel(x_left, y, self.fill_color)
                x_left -= 1
            while not self.is_edge_point(x_right, y):
                self.set_pixel(x_right, y, self.fill_color)
                x_right += 1
            x_left += 1
            x_right -= 1
            for xi in range(x_left, x_right + 1):
                if self.is_free(xi, y - 1, x_left):
                    stack.append((xi, y - 1))
                if self.is_free(xi, y + 1, x_left):
                    stack.append((xi, y + 1))

    def weighted_filter_3x3(self):
        kernel = np.array([[1, 2, 1],
                           [2, 4, 2],
                           [1, 2, 1]], dtype=np.float32)

        filtered = np.copy(self.buffer)
        h, w, _ = self.buffer.shape
        for y in range(1, h - 1):
            for x in range(1, w - 1):
                region = self.buffer[y - 1:y + 2, x - 1:x + 2]
                for c in range(3):
                    value = np.sum(region[:, :, c] * kernel) / kernel.sum()
                    filtered[y, x, c] = np.clip(value, 0, 255)
        self.buffer = filtered

    def resize_buffer(self, new_w, new_h):
        new_buffer = np.full((new_h, new_w, 3), 255, dtype=np.uint8)
        h = min(new_h, self.buffer.shape[0])
        w = min(new_w, self.buffer.shape[1])
        new_buffer[:h, :w] = self.buffer[:h, :w]
        self.buffer = new_buffer

polygons = Polygon()
```

```
114 def main():
115     if not glfw.init():
116         return
117     window = glfw.create_window(WINDOW_WIDTH, WINDOW_HEIGHT, "Lab4", None,
        None)
118     if not window:
119         glfw.terminate()
120         return
121     glfw.make_context_current(window)
122     glfw.set_key_callback(window, key_callback)
123     glfw.set_mouse_button_callback(window, mouse_button_callback)
124     glfw.set_framebuffer_size_callback(window, framebuffer_size_callback)
125 
126     glEnable(GL_TEXTURE_2D)
127     texture = glGenTextures(1)
128     while not glfw.window_should_close(window):
129         glfw.poll_events()
130 
131         glBindTexture(GL_TEXTURE_2D, texture)
132         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST
        )
133         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
        GL_NEAREST)
134         glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, polygons.buffer.shape[1], polygons.
        buffer.shape[0],
135                      0, GL_RGB, GL_UNSIGNED_BYTE, polygons.buffer)
136 
137         glClear(GL_COLOR_BUFFER_BIT)
138         glClearColor(1.0, 1.0, 1.0, 1.0)
139         glLoadIdentity()
140 
141         glBegin(GL_QUADS)
142         glTexCoord2f(0, 0); glVertex2f(-1, -1)
143         glTexCoord2f(1, 0); glVertex2f(1, -1)
144         glTexCoord2f(1, 1); glVertex2f(1, 1)
145         glTexCoord2f(0, 1); glVertex2f(-1, 1)
146         glEnd()
147 
148         glfw.swap_buffers(window)
149     glfw.destroy_window(window)
150     glfw.terminate()
151 
152 def key_callback(window, key, scancode, action, mods):
153     if action == glfw.PRESS:
154         if key == glfw.KEY_F:
155             polygons.weighted_filter_3x3()
```

```
156             if key == glfw.KEY_C:
157                 polygons.clear_buffer()
158 
159 def mouse_button_callback(window, button, action, mods):
160     if action == glfw.PRESS:
161         x, y = glfw.get_cursor_pos(window)
162         y = WINDOW_HEIGHT - int(y)
163         if button == glfw.MOUSE_BUTTON_LEFT:
164             polygons.add_point(int(x), int(y))
165             if len(polygons.points) > 1:
166                 polygons.bresenham(polygons.points[-2], polygons.points[-1])
167         if button == glfw.MOUSE_BUTTON_RIGHT:
168             if len(polygons.points) > 2:
169                 polygons.bresenham(polygons.points[-1], polygons.points[0])
170             polygons.fill_with_zatravka(int(x), int(y))
171 
172 def framebuffer_size_callback(window, width, height):
173     global WINDOW_WIDTH, WINDOW_HEIGHT, polygons
174     WINDOW_WIDTH = width
175     WINDOW_HEIGHT = height
176     polygons.resize_buffer(width, height)
177     glViewport(0, 0, width, height)
178 
179 main()
```

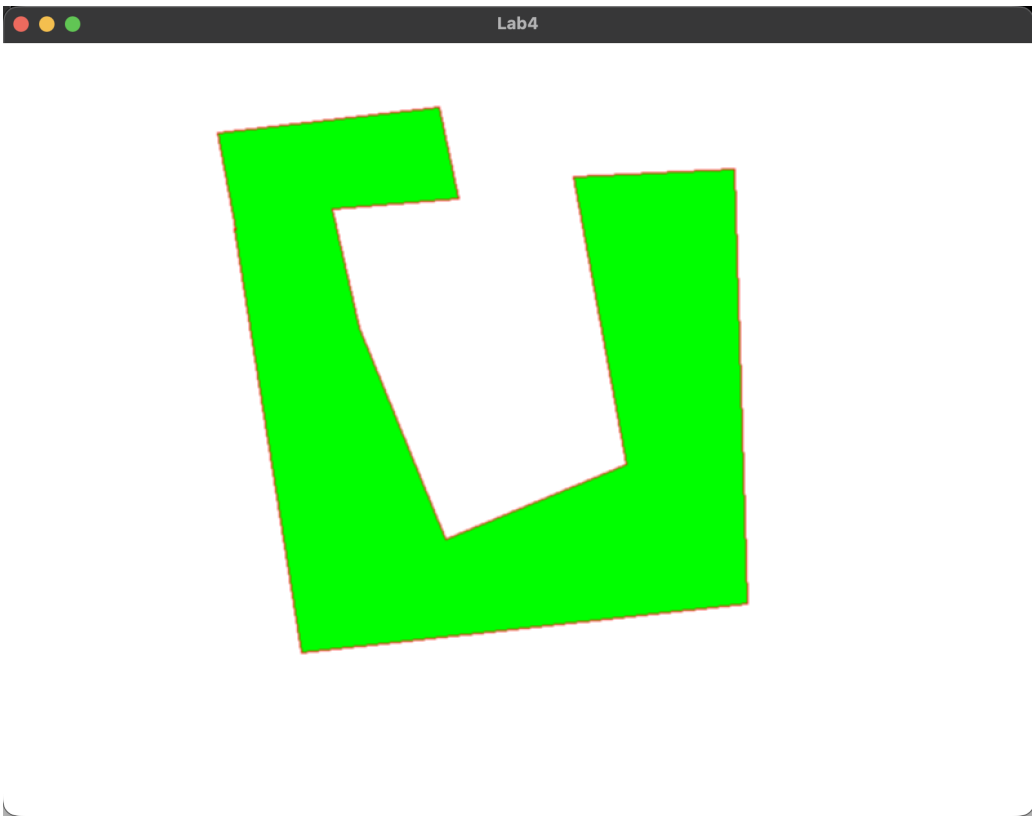


Рис. 1 — Результат работы

4 Вывод

В ходе работы реализованы алгоритм Брезенхема, построчное заполнение с затравкой и фильтр взвешенного усреднения 3×3. Программа позволяет интерактивно строить и заливать многоугольники, очищать экран, применять фильтр и изменять размер окна.